# Complexity Comparison: SC and GSC

We consider a sparse adjacency matrix $W$ with $N$ vertices and $m$ stored edges. Under the classical sparse-graph convention, one has $N = O(m)$, so $O(m + N)$ and $O(m)$ are equivalent.
We compare the time complexity of a single run of the GSC algorithm and the classical SC algorithm on $W$. The implementation and experiments considered below use the normalized Laplacian path; however, the preprocessing complexity analysis given here remains valid for both normalized and unnormalized Laplacians.

## GSC overhead

The additional GSC-specific operations are the computation of the measure $\nu_{t,\alpha}$ and the assembly of the generalized Laplacian $L_{\nu_{t,\alpha}}$ or its normalized counterpart $\bar{L}_{\nu_{t,\alpha}}$. The GSC measure used in the implementation is computed by repeated sparse multiplication with the transition matrix $P = D_{\text{out}}^{-1}W$, without ever forming the dense power $P^t$ explicitly.

**Algorithm 1: $(\alpha, t)$-dependent measure computation**

**procedure** COMPUTE-MEASURE($W$, $t$, $\alpha$)
  $P \leftarrow D_{\text{out}}^{-1}W$
  $v_0 \leftarrow \frac{1}{N}1_{1\times N}$
  **for** $\ell = 0, 1, ..., t-1$ **do**
    $v_{\ell+1} \leftarrow v_\ell P$
  **end**
  $\nu_t \leftarrow v_t^T$
  $\nu_{t,\alpha}(i) \leftarrow \nu_{t(i)}^{\alpha}$
  **return** $\nu_{t,\alpha}$
**end**

Since $P$ has the same sparsity as $W$, constructing it costs $O(m + N)$. Each update $v_{\ell+1} = v_\ell P$ is a sparse vector-matrix multiplication with cost $O(m)$, hence the $t$ iterations cost $O(tm)$. The final componentwise power and normalization cost $O(N)$. Therefore the total complexity of the measure computation is $O(tm + m + N)$, which is $O(m)$ for fixed $t$ under the classical sparse-graph convention.

## Generalized Laplacians building cost and sparsity

We now prove that the generalized Laplacian has the same sparsity order as the classical undirected Laplacian, and the asymptotic cost of building them is the same. Let

$$W_{\text{sym}} = \frac{1}{2}(W + W^T) \quad \text{and} \quad L_{\text{sym}} = D_{\text{sym}} - W_{\text{sym}}$$

On the GSC side, recall that the generalized Laplacian is

$$L_\nu = D_{\nu+\xi} - (D_\nu P + P^T D_\nu),$$

It is easy to see that both Laplacians have the same sparsity order as $W_{\text{sym}}$. Indeed, left and right multiplication by diagonal matrices only rescales existing entries and cannot create new off-diagonal nonzeros. Hence $L_{\text{sym}}$ and $L_{\nu_{t,\alpha}}$ store only diagonal terms together with entries corresponding to the support of $W$ and $W^T$, so their storage and assembly costs remain linear in the number of graph edges, i.e. $O(m)$. The same statement holds for their normalized counterparts.

In the normalized implementation path considered here, both the standard Laplacian built from $W_{\text{sym}}$ and the normalized generalized Laplacian are symmetric. Therefore the same symmetric

ARPACK routine is used for the spectral decomposition of the benchmarked SC and GSC variants, with the same asymptotic model for the spectral step.

Finally, we conclude that, for fixed $t$, the preprocessing overhead of a single run of GSC over SC is linear in the number of edges in the graph.

### Spectral Decomposition

In practice, the spectral step is carried out by `eigsh`, which wraps ARPACK's implicitly restarted Lanczos method. In the normalized scikit-learn path benchmarked here, it is used in shift-invert mode, so the implementation-level cost depends on sparse factorizations, repeated linear solves, restart parameters, and convergence behavior rather than on a single fixed formula [1], [2], [3].

For this reason, we do not claim a clean implementation-level complexity law for the spectral step. The important point for the present comparison is that the benchmarked SC and GSC variants use the same normalized spectral routine, and therefore share the same dominant spectral cost [1], [2], [3].

### K-Means

Finally, the label extraction step is performed by `kmeans` on the $N \times r$ spectral embedding. The scikit-learn documentation gives the average complexity of $k$-means as $O(knT)$, where $k$ is the number of clusters, $n$ the number of samples, and $T$ the number of Lloyd iterations [4]. In the present setting, both the embedding dimension and the number of clusters are $r$, and with $s$ restarts and $I$ Lloyd iterations this yields the model $O(sINr^2)$. This term is the same for SC and GSC, and is $O(N)$ for fixed $r$, $s$, and $I$.

### Full Single-Run Complexity

At the level of a single run, SC and GSC have the same complexity up to the additional GSC preprocessing term analyzed above. Thus the only systematic difference between the two methods is the cost of building the measure and the generalized Laplacian, while the spectral and labeling steps are shared.

## Experimental results

The experiments are carried out on sparse DISBM networks with expected degree of order $\Theta(\log N)$, hence $m = \Theta(N \log N)$ asymptotically. We first validate the part of the theory that is proved explicitly, namely the preprocessing overhead. Figure 1 shows that the preprocessing cost of both SC and GSC is consistent with linear sparse scaling in $m$, with a larger constant for GSC. We then compare the full algorithms in Figure 2. SC and GSC exhibit the same empirical growth, and their end-to-end runtime difference remains small compared with the total cost. Figure 3 explains this behavior: the eigensolver rapidly dominates the runtime, while the preprocessing terms become negligible. Thus the experiments support both claims needed here: the preprocessing analysis is validated directly, and the full algorithms have the same effective scaling because they share the same dominant spectral step.

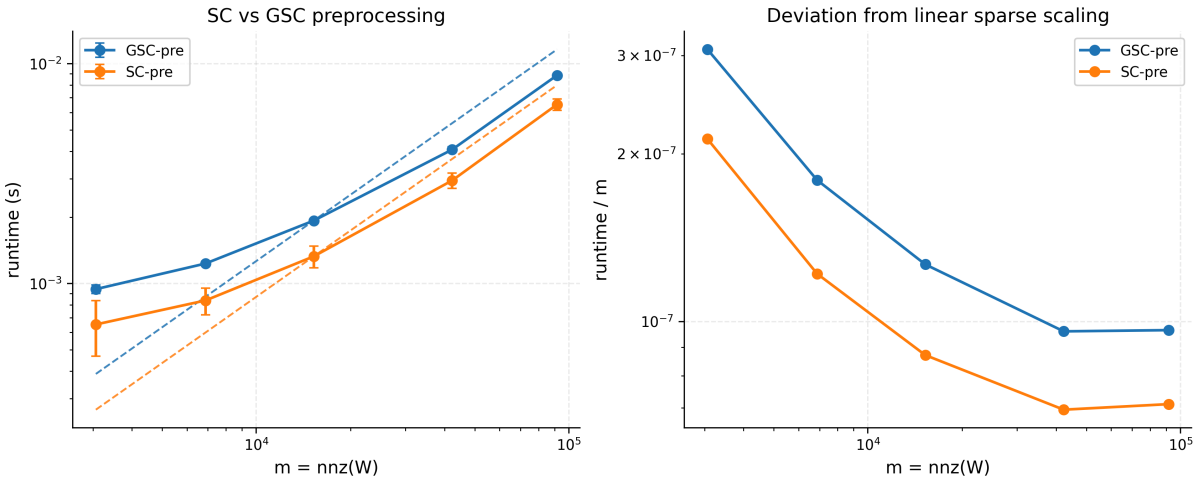


Figure 1: Preprocessing-only benchmark on sparse precomputed DISBM networks. The SC and GSC preprocessing costs scale linearly with the sparse graph size up to constants, with GSC carrying the larger constant factor predicted by the theory.

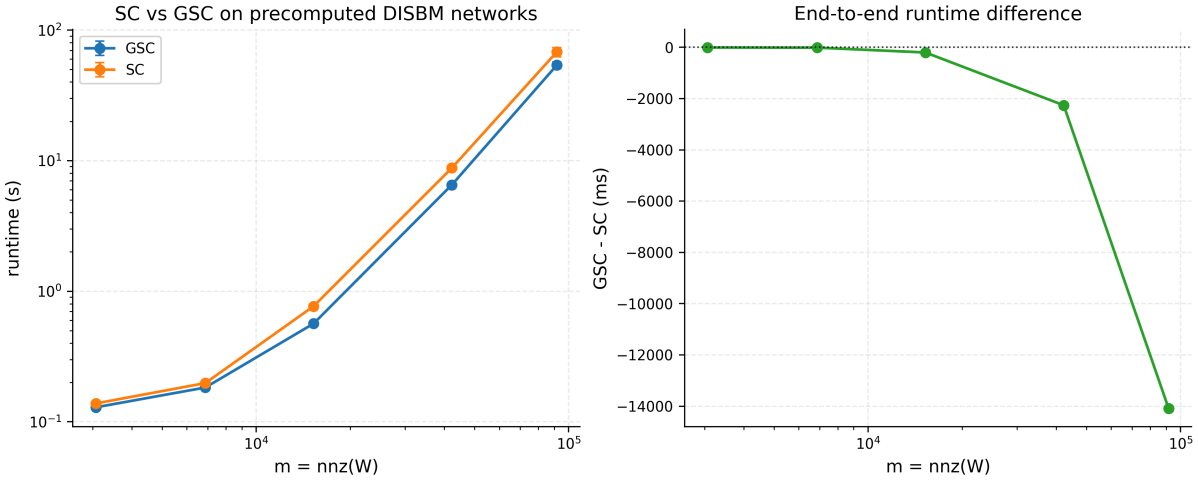


Figure 2: Single-run SC and GSC runtimes on sparse precomputed DISBM networks with expected degree of order $\Theta(\log N)$. The left panel shows that the two methods have the same empirical growth, while the right panel reports the end-to-end runtime difference GSC − SC.

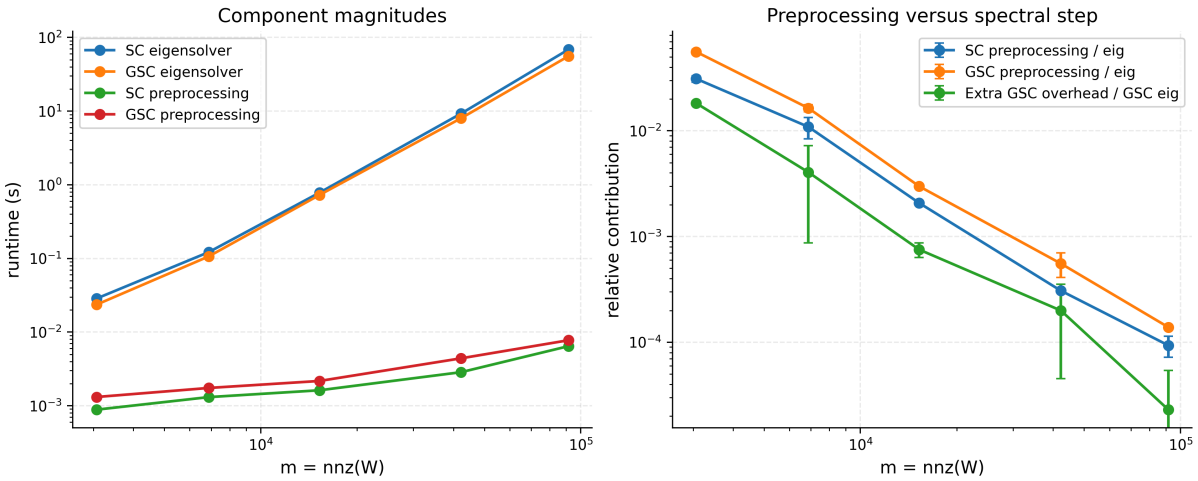


Figure 3: Component dominance on sparse precomputed DISBM networks. The eigensolver rapidly dominates the runtime, while both SC and GSC preprocessing terms become negligible in relative magnitude.

# Bibliography

[1] SciPy developers, "Sparse eigenvalue problems with ARPACK — SciPy tutorial." Accessed: Mar. 06, 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/tutorial/arpack.html

[2] SciPy developers, "eigsh — SciPy documentation." Accessed: Mar. 06, 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.eigsh.html

[3] R. B. Lehoucq, D. C. Sorensen, and C. Yang, *ARPACK Users' Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods*. Philadelphia: SIAM, 1998. doi: 10.1137/1.9780898719628.

[4] scikit-learn developers, "KMeans — scikit-learn documentation." Accessed: Mar. 06, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html